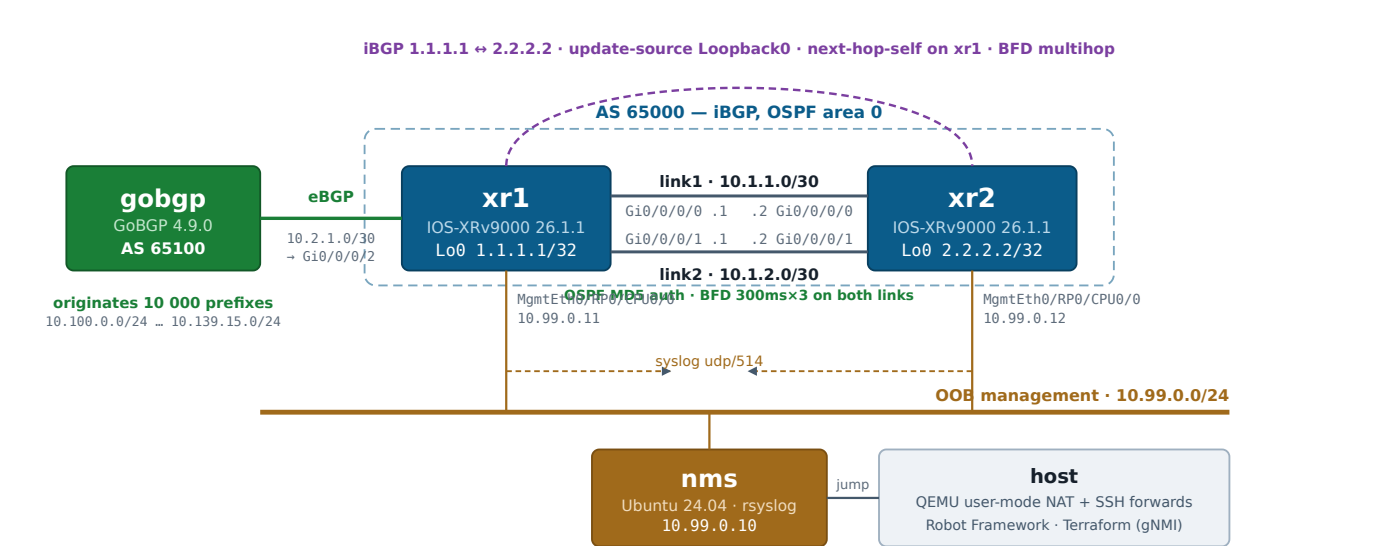


Cisco IOS-XRv9000 Lab — Topology & Testbed

Four QEMU/KVM guests on one host, no root required · OSPF + iBGP core with BFD, external eBGP speaker, out-of-band management · configured with Cisco Network as Code · github.com/dcantor/IOS-XR-NAC

Lab environment only. Credentials are committed in plaintext and transport security is disabled throughout, deliberately — see the repository README. Do not reuse either in a production network. Figures here are read from the live lab and its committed configuration.

TOPOLOGY



Solid = data plane. Dashed purple = iBGP session over loopbacks. Dashed orange = syslog. The routers' management ports exist only on the OOB network; the host reaches them through **nms**.

NODES

NODE	ROLE	IMAGE	VCPU	RAM
xr1	Router, eBGP edge	IOS-XRv9000 26.1.1	4	20 GiB
xr2	Router	IOS-XRv9000 26.1.1	4	20 GiB
nms	Syslog server, jump host	Ubuntu 24.04 minimal	1	1 GiB
gobgp	External BGP speaker	Ubuntu 24.04 + GoBGP 4.9.0	1	1 GiB

BGP

SESSION	TYPE	LOCAL AS	PEER AS	PREFIXES
1.1.1.1 ↔ 2.2.2.2	iBGP	65000	65000	10 000
10.2.1.1 ↔ 10.2.1.2	eBGP	65000	65100	10 000

iBGP peers loopback-to-loopback (update-source Loopback0), so the session rides the OSPF-learned /32 and survives either link failing. xr1 applies next-hop-self so xr2 can resolve the external prefixes.

HOST REQUIREMENTS

RESOURCE	NEEDED	NOTE
RAM	≈ 42 GiB	20 GiB per router is not negotiable — below ~18 GiB the XRv9k host OS wedges during boot with nothing logged.
Disk	≈ 15 GiB	Copy-on-write overlays; the 2 GB base image is never written to.
Virtualisation	/dev/kvm	Nested virtualisation required — XRv9k runs XR inside a nested VM.
Firmware	OVMF (UEFI)	The image is UEFI-only; its protective MBR carries no boot code.
Privileges	none	No bridges, no tap devices, no root.

ADDRESSING

SEGMENT	SUBNET	XR1	XR2	OTHER END / NOTES
Loopback0	/32	1.1.1.1	2.2.2.2	Router identity; advertised into OSPF as passive, and the iBGP peering address
link1 — Gi0/0/0/0	10.1.1.0/30	10.1.1.1	10.1.1.2	Router-to-router
link2 — Gi0/0/0/1	10.1.2.0/30	10.1.2.1	10.1.2.2	Router-to-router
eBGP — Gi0/0/0/2	10.2.1.0/30	10.2.1.1	—	gobgp at 10.2.1.2; xr1 only
OOB — MgmtEth0/RP0/CPU0/0	10.99.0.0/24	10.99.0.11	10.99.0.12	nms at 10.99.0.10, which also collects syslog on udp/514
Loopback98 (NaC-managed)	/32	10.98.1.1	10.98.2.1	Created by Terraform, not by day-0 config
Injected prefixes	10.100.0.0/24 ... 10.139.15.0/24	10 000 × /24		Originated by gobgp; prefix <i>n</i> is 10.<100+n/256>.<n%256>.0/24
OSPF router-id / area	area 0	1.1.1.1	2.2.2.2	Process CORE; both links point-to-point with MD5 key id 1
gNMI	tcp/57400	no-tls		How Terraform reaches the routers, tunnelled via nms

ROUTING PROTOCOLS

PROTOCOL	WHERE	DETAIL
OSPF CORE, area 0	both links + Loopback0 (passive)	Point-to-point. Each router learns the other's /32 over both links, giving two equal-cost paths — visible in CEF as a recursive load-share across Gi0/0/0/0 and Gi0/0/0/1. Both links require MD5 authentication (key id 1): no key, no adjacency.
BFD	2 single-hop + 1 multihop, per router	300 ms × 3 = 900 ms detection, against OSPF's 40 s dead interval. Single-hop on each link for OSPF; multihop between the loopbacks for iBGP, which needs <code>bfd multipath include location 0/0/CPU0</code> or it never leaves <code>BFD_MP_DOWNLOAD_NO_LC</code> . Not enabled towards gobgp — GoBGP has no BFD implementation.
iBGP AS 65000	Loopback0 ↔ Loopback0	Carries the 10 000 external prefixes from xr1 to xr2 with next-hop-self, so they arrive resolvable via 1.1.1.1. Protected by multihop BFD.
eBGP AS 65000 ↔ 65100	xr1 Gi0/0/0/2 ↔ gobgp	gobgp originates 10 000 /24s. Inbound policy validates both the prefix range and the AS path before accepting any of them.
CDP	Gi0/0/0/0, Gi0/0/0/1	Each router sees exactly its peer on each link — the check that caught an early wiring bug where frames looped back to the sender.

ROUTING POLICY

IOS-XR applies **no default policy** to an eBGP neighbour: without one the session establishes and every prefix is silently discarded. Day-0 therefore installs a permissive PASS purely to bootstrap, and Network as Code replaces the inbound half with a policy that checks two independent things.

OBJECT	ON	PURPOSE
NAC-GOBGP-ASPATH	xr1, xr2	as-path-set ios-regex '^65100\$' — an AS path of exactly one hop, AS 65100: gobgp originated it and nothing else has touched it.
NAC-GOBGP-IN	xr1 (eBGP in)	Accepts only if the destination is in the expected /24 range and the AS path matches the set. Everything else is dropped.
NAC-GOBGP-OUT	xr1 (eBGP out)	pass.
NAC-IBGP-IN	xr2 (iBGP in)	Same AS-path check applied again on what xr1 relays — defence in depth, since AS paths are not rewritten inside an AS.

Why the AS-path check matters: an eBGP peer can advertise any AS path it likes. A prefix in the right range but carrying 65100 65200 — i.e. it transited somewhere it should not have — is rejected. The test suite proves this by originating two prefixes that differ *only* in AS path and confirming just one reaches the table.

MANAGEMENT AND ACCESS

NODE	CONSOLE (TELNET)	SSH FROM HOST	QEMU MONITOR	CREDENTIALS
xr1	127.0.0.1:5101	10.99.0.11 via nms	127.0.0.1:4101	admin
xr2	127.0.0.1:5111	10.99.0.12 via nms	127.0.0.1:4111	admin
nms	127.0.0.1:5121	127.0.0.1:2261	127.0.0.1:4121	lab
gobgp	127.0.0.1:5131	127.0.0.1:2262	127.0.0.1:4131	lab

Only nms and gobgp have a host SSH forward. XRv9k has a single management port, so it cannot sit on the OOB network *and* behind a NAT forward — the host has no route to the routers. nms bridges that: it is on the OOB network and reachable from the host, so it serves as the jump host for both the CLI and the test suite.

VIRTUAL WIRING

SEGMENT	QEMU NETDEV	WHY
link1 / link2	socket, unicast UDP pairs 11001↔11002, 11003↔11004	A true point-to-point wire. Multicast sockets loop frames back to the sender, which made every router appear as its own CDP neighbour.
eBGP link	socket, UDP 11005↔11006	Same, to xr1's sixth vNIC.
OOB network	socket, mcast 230.31.0.1:11099	A shared segment for three nodes. Multicast loopback is harmless here: no link-layer neighbour protocol runs on it.
Host access	user (SLIRP) + hostfwd	nms and gobgp only.

vNIC order fixes XRv9k interface naming and cannot be varied: 1 MgmtEth0/RP0/CPU0/0 · 2 CtrlEth · 3 DevEth · 4 Gi0/0/0/0 · 5 Gi0/0/0/1 · 6 Gi0/0/0/2 (xr1 only). CtrlEth and DevEth are internal to a real chassis, but the image expects the vNICs, so they get dead-end hubs.

CONFIGURATION OWNERSHIP

LAYER	OWNS	DELIVERED BY
Day-0 configs/<node>.cfg	Bootstrap: hostname, credentials, management address, ssh, gRPC, bfd — plus the interfaces, OSPF (with MD5 auth and BFD) and iBGP the lab is built on	cvac, from a CD-ROM at first boot
Network as Code nac/iosxr.nac.yaml	Loopback98, the AS-path set, the route-policies, and which policies each BGP neighbour uses	Terraform over gNMI — netascode/nac-iosxr 0.1.1 / CiscoDevNet/iosxr 0.7.1

Day-0 must come first: it is what makes a router reachable by automation. Network as Code then reconciles what it owns — an out-of-band edit shows up as a terraform plan diff and is corrected by the next apply.

VERIFICATION

29 of 29 Robot Framework tests passing (77 s). Each run writes to its own timestamped directory under test_results/; sessions are tunnelled through nms, the same path an operator uses.

TAG	COVERS
version, config	Software release and that day-0 config actually applied
cdp	Each router sees exactly its peer on each link, on the matching remote port
ospf, ecmp	One FULL neighbour per link; the peer loopback learned over two equal-cost paths
ospf, auth	Message-digest authentication active on each link, key id 1
bfd	Both single-hop sessions and the multihop iBGP session up, with OSPF and BGP reporting them as clients
bgp, ibgp	Sessions established; peering uses the loopback addresses; all 10 000 prefixes relayed
nms, oob, ssh, syslog	OOB reachability, nms logging in to each router, syslog delivery per router
gobgp, prefixes	All 10 000 originated, accepted and installed
aspath	A prefix with the wrong AS path is denied — end to end
nac	NaC-managed config matches the model, and day-0 config survived the apply